מסלול לימוד · GPU ו-NVLink · גרסה ללא אינטרנט

להכיר את כל הרכיבים החשובים של ה-GPU, צעד אחר צעד

9 פרקים, 43 שלבים ו-60 רכיבים. **כל שלב הוא שאלה אמריקאית ואחריה הסבר מפורט.**

כל שאלה בעמוד משלה. לחשוב על תשובה, ואז לדפדף לעמוד הבא: שם התשובה וההסבר. כל מונח מסומן הוא קישור להגדרה שלו במילון בסוף, ושם יש קישורים חזרה לשלבים.

פרק 1: התמונה הגדולה

פרק 2: איך עבודה נכנסת ל-GPU

פרק 3: החישוב: מ-GPC ועד נתיב

פרק 4: הזיכרון

פרק 5: הקושחה על ה-GPU

פרק 6: NVLink: התמונה

פרק 7: בתוך הקישור: הפרוטוקול

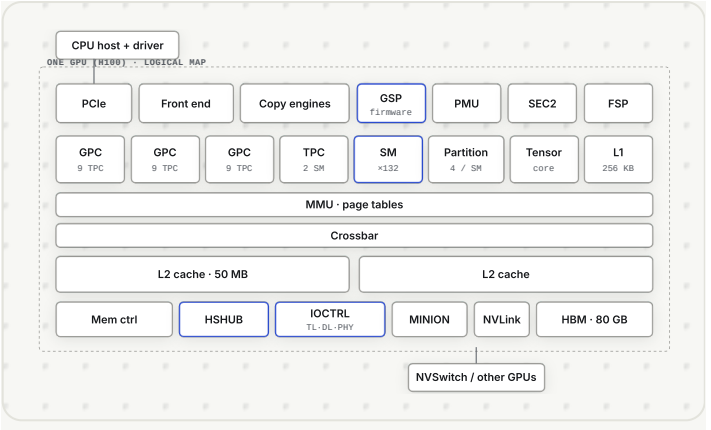
פרק 8: הקמה וניהול: איפה הקוד רץ

פרק 9: NVSwitch ומערכות גדולות

מילון הרכיבים

לפני שמתחילים

מפת המערכת: כל בלוק הוא קישור להגדרה שלו



מפה לוגית, לא תוכנית אמיתית של השבב. בקרה למעלה, חישוב באמצע, זיכרון וקישורים למטה.

התמונה הגדולה

מה זה GPU, ממה הוא בנוי, ולמה צריך לחבר כמה GPU-ים.

רכיבים בפרק: SM · HBM · GSP · NVLink · PCIe

1/43 שלב

מה ההבדל העיקרי בין GPU ל-CPU?

א. ל-GPU יש אלפי נתיבי חישוב פשוטים שמריצים את אותה הוראה על הרבה נתונים

ב. ל-GPU יש כמה ליבות חזקות, וכל אחת מריצה משימה אחרת

ג. ה-GPU מריץ את מערכת ההפעלה במקום ה-CPU

ד. ה-GPU מהיר רק כי השעון שלו מהיר יותר

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

ל-CPU יש מעט ליבות חזקות, וכל אחת עושה עבודה אחרת. ל-GPU יש אלפי נתיבי חישוב פשוטים: ב-H100 יש 132 SM, ובכל אחד 128 נתיבים, כלומר כ-16,900. כל 32 נתיבים מבצעים יחד את אותה הוראה, וכל אחד עובד על נתון אחר. הקבוצה הזו נקראת Warp. לכן GPU מהיר כשאותו צעד חוזר על המון נתונים, כמו כפל מטריצות ב-AI. המהירות באה מהמקביליות, לא מהשעון.

למה לא האחרות

"כמה ליבות חזקות" מתאר CPU. מערכת ההפעלה והדרייבר רצים על ה-CPU. השעון של GPU בדרך כלל איטי יותר משל CPU.

רכיבים לחזור עליהם

SM

הליבה של ה-GPU, שמריצה את ה-threads.

FP32 lanes

יחידות שמבצעות חישוב רגיל במספרים.

Warp

32 threads שמריצים תמיד את אותה הוראה יחד.

מה זה SM?

א יחידת זיכרון גדולה ליד השבב

ב בקר קטן שמריץ את הקושחה

ג הממשק שמחבר את ה-GPU ל-CPU

ד הליבה של ה-GPU, שמריצה את ה-threads

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

ה-SM (Streaming Multiprocessor) הוא הליבה האמיתית של ה-GPU. ב-H100 SXM5 פעילים 132. הם מקובצים בזוגות בתוך TPC, וה-TPC-ים בתוך GPC. בתוך כל SM יש 4 partitions ו-L1 / shared memory משותף. בהמשך המסלול נפתח את ה-SM לגמרי.

למה לא האחרות

זיכרון גדול ליד השבב הוא HBM. הבקר שמריץ קושחה הוא GSP. הממשק ל-CPU הוא PCIe.

רכיבים לחזור עליהם

SM

הליבה של ה-GPU, שמריצה את ה-threads.

GPC

קבוצה גדולה של יחידות חישוב.

TPC

זוג SM-ים.

איפה נמצא הזיכרון הגדול ביותר של ה-GPU?

א L1

ב L2

ג HBM

ד Registers

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

HBM הוא שבבי זיכרון שמוערמים ליד ה-GPU: 80 GB בקצב 3.35 TB/s ב-H100 SXM5. ככל שזיכרון רחוק יותר מהנתיבים, הוא גדול ואיטי יותר: registers ו-L1 / shared memory הם 256 KB לכל SM, ה-L2 cache הוא 50 MB לכל השבב, וה-HBM הוא 80 GB. ה-memory controllers הם שמדברים עם ה-HBM.

למה לא האחרות

L1 ו-registers הם הקטנים והמהירים ביותר. L2 גדול מהם, אבל קטן פי אלפים מ-HBM.

רכיבים לחזור עליהם

HBM

הזיכרון הגדול של ה-GPU.

L2 cache

זיכרון מטמון אחד לכל השבב.

L1 / shared memory

זיכרון מהיר בתוך כל SM.

Memory controller

יחידה שקוראת וכותבת ל-HBM.

למה GPU-ים צריכים NVLink?

א כדי להתחבר למסך

ב כי מודלים גדולים מתחלקים בין הרבה GPU-ים שמחליפים נתונים כל הזמן, ו-PCIe איטי מדי לזה

ג כי PCIe מהיר מדי

ד רק כדי לחסוך חשמל

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

מודל AI גדול לא נכנס ב-GPU אחד, אז העבודה מתחלקת וה-GPU-ים מחליפים תוצאות כל הזמן. PCIe Gen5 נותן כ-128 GB/s בשני הכיוונים. NVLink 4 ב-H100 נותן 900 GB/s, בערך פי 7. זה עדיין פחות מה-HBM המקומי (3,350 GB/s), ולכן תוכנה טובה שומרת את רוב התעבורה מקומית.

למה לא האחרות

NVLink לא מחבר מסכים. PCIe איטי, לא מהיר מדי. חיסכון בחשמל הוא לא הסיבה.

רכיבים לחזור עליהם

NVLink

חיבור ישיר ומהיר בין GPU-ים, או בין GPU ל-NVSwitch.

PCIe

החיבור בין ה-GPU ל-CPU.

HBM

הזיכרון הגדול של ה-GPU.

איך עבודה נכנסת ל-GPU

מהקריאה באפליקציה ועד שה-SM-ים מתחילים לעבוד, ובחזרה.

רכיבים בפרק: Driver · Pushbuffer · Doorbell · Front end · Work distributor · Copy engine · Semaphore

האפליקציה מריצה kernel. מה הדרייבר עושה כדי שה-GPU יידע על זה?

א. שולח את הקוד ישירות ל-SM-ים

ב. כותב פקודה ל-pushbuffer בזיכרון ומצלצל ב-doorbell

ג. מבקש מה-GSP להריץ את ה-kernel

ד. מעתיק את ה-kernel ל-L2

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

ה-Driver כותב את הפקודה לתוך Pushbuffer: אזור בזיכרון שמשמש "רשימת משימות" ל-GPU. אחר כך הוא כותב לרגיסטר Doorbell ב-GPU, וזה הסימן שיש עבודה חדשה. מכאן ה-GPU ממשיך לבד. ה-CPU לא מדבר עם ה-SM-ים ישירות.

למה לא האחרות

ה-CPU לא שולח קוד ישירות ל-SM-ים. ה-GSP מנהל את השבב, אבל לא מריץ kernels של משתמש. ה-L2 הוא מטמון, לא תור פקודות.

רכיבים לחזור עליהם

Driver

מודול בקרנל של לינוקס שמנהל את ה-GPU מצד ה-CPU.

Pushbuffer

אזור בזיכרון שבו הדרייבר כותב רשימת פקודות ל-GPU.

Doorbell

רגיסטר שהדרייבר כותב אליו.

איזו יחידה קוראת את רשימת הפקודות מה-pushbuffer?

א Front end

ב Copy engine

ג L2 cache

ד MMU

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

ה-Front end שולף את הפקודות מה-Pushbuffer, מפענח אותן, ושולח כל אחת ליחידה הנכונה: עבודת חישוב ל-Work distributor, והעתקות ל-Copy engine.

למה לא האחרות

Copy engine מעתיק נתונים ולא קורא פקודות. L2 cache הוא מטמון. MMU מתרגם כתובות.

רכיבים לחזור עליהם

Front end

היחידה בראש ה-GPU שקוראת את רשימת הפקודות.

Work distributor

החלק של ה-front end שמחלק עבודת חישוב.

Copy engine

מנוע שמעתיק בלוקים גדולים של זיכרון.

מי מחליט איזה thread block ירוץ על איזה SM?

א המתכנת

ב ה-L2

ג ה-PMU

ד ה-Work distributor

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

ה-Work distributor (ב-NVIDIA גם GigaThread engine) הוא חלק מה-front end. הוא מקבל Grid ושולח כל Thread block ל-SM שיש בו מקום. כשבלוק מסתיים, הוא שולח את הבא. המתכנת קובע כמה blocks יש, אבל לא איפה הם ירוצו.

למה לא האחרות

המתכנת לא בוחר SM. ה-L2 הוא זיכרון. ה-PMU מנהל חשמל ושעונים.

רכיבים לחזור עליהם

Work distributor

החלק של ה-front end שמחלק עבודת חישוב.

Thread block

קבוצה של עד 1,024 threads.

Grid

כל ה-thread blocks של הרצה אחת של Kernel.

מה התפקיד של Copy engine?

א לחשב כפל מטריצות

ב לתרגם כתובות

ג להעתיק בלוקים גדולים של נתונים (DMA) בלי להעסיק את ה-SM-ים

ד לאתחל את ה-GPU

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

Copy engines הם מנועי DMA. cudaMemcpy משתמש בהם כדי להעביר קלט מזיכרון המחשב ל-HBM דרך PCIe, ותוצאות בחזרה. הם יכולים להעתיק גם ל-GPU אחר דרך NVLink. היתרון: העתקה וחישוב רצים במקביל.

למה לא האחרות

כפל מטריצות נעשה ב-tensor cores. תרגום כתובות: MMU. אתחול: הקושחה ב-GSP.

רכיבים לחזור עליהם

Copy engine

מנוע שמעתיק בלוקים גדולים של זיכרון.

HBM

הזיכרון הגדול של ה-GPU.

PCIe

החיבור בין ה-GPU ל-CPU.

איך ה-CPU יודע שה-kernel הסתיים?

א. ה-SM שולח הודעה ישירות לאפליקציה

ב. ה-CPU מחכה זמן קבוע

ג. ה-front end כותב semaphore לזיכרון, וה-CPU רואה את הערך

ד. ה-GSP מפעיל מחדש את ה-CPU

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

כשכל ה-blocks מסתיימים, ה-Front end כותב Semaphore: ערך בזיכרון שה-CPU יכול לקרוא. הדרייבר רואה את הערך ומודיע לאפליקציה. אחר כך Copy engine מחזיר את התוצאות.

למה לא האחרות

אין ל-SM ערוץ ישיר לאפליקציה. זמן קבוע לא אמין. ה-GSP לא מפעיל את ה-CPU.

רכיבים לחזור עליהם

Semaphore

כתיבה של ערך לזיכרון כדי לסמן שמשהו הסתיים.

Front end

היחידה בראש ה-GPU שקוראת את רשימת הפקודות.

החישוב: מ-GPC ועד נתיב

ההיררכיה של יחידות החישוב, ואיך הקוד שלך ממופה עליה.

רכיבים בפרק: GPC · TPC · SM · SM partition · Warp scheduler · Register file · FP32 lanes · Tensor core · L1 / shared memory · Warp · Thread block · Thread

מה הסדר הנכון של ההיררכיה, מהגדול לקטן?

א. SM ← TPC ← GPC ← partition

ב. GPC ← TPC ← SM ← partition

ג. TPC ← GPC ← partition ← SM

ד. partition ← SM ← TPC ← GPC

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

GPC מכיל 9 TPC-ים. כל TPC מכיל 2 SM-ים. כל SM מתחלק ל-4 partitions. בשבב GH100 המלא: 2 × 9 × 8 = 144 SM, ומתוכם 132 פעילים ב-H100 SXM5.

למה לא האחרות

בכל שאר האפשרויות הסדר הפוך או מעורבב. כדאי לזכור את המספרים: 8 GPC, 9 TPC בכל אחד, 2 SM בכל TPC.

רכיבים לחזור עליהם

GPC

קבוצה גדולה של יחידות חישוב.

TPC

זוג SM-ים.

SM

הליבה של ה-GPU, שמריצה את ה-threads.

SM partition

אחד מארבעת החלקים של SM.

מה יש בכל SM partition?

א) warp scheduler, register file, 32 נתיבי FP32 ו-tensor core

ב) רק זיכרון

ג) בקר זיכרון ו-HBM

ד) MINION ו-PHY

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

כל SM partition הוא כמו ליבה קטנה: Warp scheduler שבוחר warp, Register file של 64 KB, 32 נתיבי FP32 ו-Tensor core אחד. ארבעת ה-partitions חולקים 256 KB של L1 / shared memory. לכן ב-SM יש 128 נתיבים ו-4 tensor cores.

למה לא האחרות

זיכרון לבד לא מבצע חישוב. בקר זיכרון ו-HBM נמצאים בשולי השבב. MINION ו-PHY / SerDes שייכים לבלוק הקישור.

רכיבים לחזור עליהם

SM partition

אחד מארבעת החלקים של SM.

Warp scheduler

היחידה שבוחרת איזה warp ירוץ עכשיו.

Register file

האחסון המהיר ביותר על השבב.

FP32 lanes

יחידות שמבצעות חישוב רגיל במספרים.

Tensor core

יחידה שמכפילה מטריצות קטנות בצעד אחד.

כמה threads יש ב-warp אחד?

א 16

ב 64

ג 1,024

ד 32

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

Warp הוא 32 threads שתמיד מריצים את אותה הוראה. בכל מחזור, ה-Warp scheduler בוחר warp מוכן ושולח הוראה אחת ל-32 הנתיבים. אם threads ב-warp הולכים לכיוונים שונים בקוד (if/else), החומרה מריצה כל צד בנפרד, וזה מאט.

למה לא האחרות

1,024 הוא הגודל המרבי של thread block, לא של warp.

רכיבים לחזור עליהם

Warp

32 threads שמריצים תמיד את אותה הוראה יחד.

Thread

עותק אחד של ה-Kernel, שעובד על פיסת נתונים אחת.

Warp scheduler

היחידה שבוחרת איזה warp ירוץ עכשיו.

על מה רץ thread block?

א מתפזר על כל ה-SM-ים

ב על נתיב אחד

ג על ה-GSP

ד על SM אחד, ונשאר בו עד שהוא מסיים

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

יש התאמה בין התוכנה לחומרה: Grid רץ על כל ה-GPU, Thread block רץ על SM אחד ונשאר בו עד הסוף, Warp רץ על SM partition אחד, ו-Thread רץ על נתיב אחד. בגלל שבלוק נשאר ב-SM אחד, ה-threads שלו יכולים לחלוק shared memory ולהסתנכרן.

למה לא האחרות

בלוק לא מתפזר בין SM-ים. נתיב מריץ thread אחד. ה-GSP לא מריץ קוד משתמש.

רכיבים לחזור עליהם

Thread block

קבוצה של עד 1,024 threads.

SM

הליבה של ה-GPU, שמריצה את ה-threads.

L1 / shared memory

זיכרון מהיר בתוך כל SM.

למה ה-warp scheduler מחזיק הרבה warps בו-זמנית?

א. כדי לחסוך רגיסטרים

ב. כי כל warp רץ על נתיב אחד

ג. כדי שכאשר warp מחכה לזיכרון, הוא יעבור מיד לאחר והנתיבים לא יעמדו

ד. כדי להאט את ה-SM

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

גישה לזיכרון לוקחת מאות מחזורי שעון. במקום לחכות, ה-Warp scheduler עובר ל-warp אחר שמוכן. המעבר כמעט חינמי, כי הרגיסטרים של כל ה-warps כבר יושבים ב-Register file. לכן מריצים על GPU הרבה יותר threads ממספר הנתיבים.

למה לא האחרות

הרבה warps צורכים יותר רגיסטרים, לא פחות. warp רץ על 32 נתיבים.

רכיבים לחזור עליהם

Warp scheduler

היחידה שבוחרת איזה warp ירוץ עכשיו.

Register file

האחסון המהיר ביותר על השבב.

Warp

32 threads שמריצים תמיד את אותה הוראה יחד.

מה התפקיד של tensor core?

א לנהל את צריכת החשמל

ב להכפיל מטריצות קטנות בצעד אחד

ג לשמור את ה-page tables

ד לשלוח flits

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

Tensor core מבצע כפל של מטריצות קטנות בצעד אחד, הרבה יותר מהר מ-נתיבי FP32 רגילים. מודלי AI בנויים בעיקר מכפל מטריצות, ולכן רוב העבודה שלהם רצה על tensor cores. יש 4 בכל SM.

למה לא האחרות

חשמל: PMU. page tables: בזיכרון, ה-MMU קורא אותן. flits: שכבות הקישור.

רכיבים לחזור עליהם

Tensor core

יחידה שמכפילה מטריצות קטנות בצעד אחד.

SM

הליבה של ה-GPU, שמריצה את ה-threads.

פרק 4 מתוך 9

הזיכרון

איך SM מגיע לנתונים, ומי מתרגם, מחבר ומאחסן בדרך.

רכיבים בפרק: MMU · Page table · Crossbar · L1 / shared memory · L2 cache · Memory controller · HBM

16/43 שלב

איזו יחידה מתרגמת את הכתובות שהתוכנית משתמשת בהן לכתובות אמיתיות?

א MMU

ב Crossbar

ג Front end

ד HSHUB

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

ה-MMU מתרגם כתובת וירטואלית למיקום אמיתי לפי page tables שה-Driver בנה, וחוסם גישה אסורה. חשוב ל-NVLink: רשומה יכולה להצביע לזיכרון של GPU אחר.

למה לא האחרות

Crossbar מחבר יחידות ולא מתרגם. Front end קורא פקודות. HSHUB מנתב לקישורים, אבל רק אחרי התרגום.

רכיבים לחזור עליהם

MMU

יחידה שמתרגמת כתובות.

Page table

טבלה שממפה כתובות וירטואליות למיקום אמיתי.

מה התפקיד של ה-crossbar?

א. לחבר כל SM לכל חלק של ה-L2

ב. לחבר GPU-ים שונים

ג. לחבר את ה-GPU ל-CPU

ד. לנתב packets ב-NVSwitch

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

ה-Crossbar הוא רשת על השבב בין ה-GPC-ים ל-L2 cache. ה-L2 מחולק לחלקים (slices), וכל אחד אחראי על חלק מהכתובות. ה-crossbar מאפשר לכל SM להגיע לכל slice.

למה לא האחרות

GPU-ים מתחברים ב-NVLink. ה-CPU מתחבר ב-PCIe. ניתוב ב-NVSwitch: NPORT.

רכיבים לחזור עליהם

Crossbar

הרשת שמחברת את ה-SM-ים לזיכרון.

L2 cache

זיכרון מטמון אחד לכל השבב.

SM

הליבה של ה-GPU, שמריצה את ה-threads.

דרך איזו יחידה עוברת כל גישה לזיכרון, גם מקומית וגם מ-GPU אחר?

א. רק L1

ב. Copy engine

ג. PCIe

ד. L2 cache

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

המסלול: SM ← L1 / shared memory ← Crossbar ← L2 ← Memory controller. אם הנתון לא ב-L2, בקשה מקומית ממשיכה ל-cache ול-HBM, ובקשה לזיכרון של GPU אחר ממשיכה ל-HSHUB ול-NVLink. בצד השני, ה-GPU השכן עונה מה-L2 שלו.

למה לא האחרות

L1 שייך ל-SM אחד. Copy engine ו-PCIe לא במסלול של load רגיל.

רכיבים לחזור עליהם

L2 cache

זיכרון מטמון אחד לכל השבב.

L1 / shared memory

זיכרון מהיר בתוך כל SM.

Crossbar

הרשת שמחברת את ה-SM-ים לזיכרון.

Memory controller

יחידה שקוראת וכותבת ל-HBM.

HSHUB

הרכזת שמחברת את ה-L2 לקישורים.

מה הסדר של רמות הזיכרון, מהמהירה לאיטית?

א registers ← L1 ← L2 ← HBM

ב L1 ← HBM ← registers ← L2

ג HBM ← L2 ← L1 ← registers

ד L2 ← registers ← HBM ← L1

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

Registers הם הכי מהירים ופרטיים לכל thread. L1 / shared memory הוא 256 KB לכל SM. L2 cache הוא 50 MB לכל השבב. HBM הוא 80 GB. כל רמה גדולה ואיטית יותר מהקודמת. קוד טוב שומר נתונים ברמות העליונות כמה שאפשר.

למה לא האחרות

שאר האפשרויות מערבבות את הסדר.

רכיבים לחזור עליהם

Register file

האחסון המהיר ביותר על השבב.

L1 / shared memory

זיכרון מהיר בתוך כל SM.

L2 cache

זיכרון מטמון אחד לכל השבב.

HBM

הזיכרון הגדול של ה-GPU.

הקושחה על ה-GPU

הבקרים הקטנים שמאתחלים ומנהלים את השבב. כאן יעבוד מהנדס/ת קושחה.

רכיבים בפרק: GSP · GSP-RM · PMU · SEC2 · FSP · Driver

מה זה GSP?

- **א** סוג של מטמון
- **ב** מעבד RISC-V בתוך ה-GPU שמריץ קושחה שמאתחלת ומנהלת את השבב
- **ג** ספרייה של CUDA
- **ד** שם נוסף ל-SM

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

GSP (GPU System Processor) הוא מעבד RISC-V קטן בתוך ה-GPU. הוא מריץ GSP-RM, החלק של הדרייבר של NVIDIA שעבר לרוץ על ה-GPU. לצידו יש בקרים נוספים: PMU, SEC2 ו-FSP.

למה לא האחרות

GSP הוא מעבד, לא זיכרון ולא ספרייה. ה-SM מריץ קוד משתמש; ה-GSP מריץ קוד ניהול.

רכיבים לחזור עליהם

GSP

מעבד RISC-V בתוך ה-GPU.

GSP-RM

החלק של הדרייבר שעבר לרוץ כקושחה על ה-GSP.

איזה בקר אחראי על מה?

א PMU: אבטחה · SEC2: חשמל · FSP: חישוב

ב PMU: חשמל ושעונים · SEC2: משימות אבטחה · FSP: אתחול מאובטח

ג FSP: PCIe · SEC2: HBM · PMU: NVLink

ד כולם עושים את אותו דבר

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

PMU מנהל שעונים ומתחים לפי עומס וחום. SEC2 מטפל במשימות אבטחה. FSP, ב-Hopper, בודק שהקושחה אמיתית לפני שהיא רצה (root of trust). ה-GSP מריץ את רוב הניהול. השמות מופיעים בקוד הפתוח של NVIDIA; התיעוד הפנימי של הצוות הוא המקור המדויק.

למה לא האחרות

שאר האפשרויות מחליפות את התפקידים.

רכיבים לחזור עליהם

PMU

בקר קטן לניהול צריכת חשמל.

SEC2

בקר קטן למשימות אבטחה.

FSP

בקר אבטחה ב-Hopper.

איך הדרייבר ב-CPU ו-GSP-RM עובדים יחד?

א. הדרייבר מחליט ושולח RPC דרך PCIe, ו-GSP-RM מבצע על החומרה

ב. הם לא מדברים

ג. GSP-RM מריץ את האפליקציה

ד. הדרייבר מריץ את הקושחה על ה-CPU

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

הדרייבר מחולק לשניים: חלק שרץ על ה-CPU (Driver), ו-GSP-RM שרץ על ה-GSP. ה-CPU שולח בקשות (RPC) דרך PCIe, ו-GSP-RM מתכנת רגיסטרים בשבב. בקוד הפתוח רואים את זה בפונקציה knvlinkExecGspRmRpc. איזו עבודה רצה בכל צד משתנה לפי גרסה ודור.

למה לא האחרות

הם מדברים כל הזמן. GSP-RM לא מריץ אפליקציות. הקושחה רצה על ה-GPU, לא על ה-CPU.

רכיבים לחזור עליהם

Driver

מודול בקרנל של לינוקס שמנהל את ה-GPU מצד ה-CPU.

GSP-RM

החלק של הדרייבר שעבר לרוץ כקושחה על ה-GSP.

PCIe

החיבור בין ה-GPU ל-CPU.

NVLink: התמונה

מה הקישור מעביר, כמה מהר, ואיך load מגיע ל-GPU אחר.

רכיבים בפרק: NVLink · Link · Sub-link · HSHUB · MMU · NVSwitch

מה רוחב הפס הכולל של NVLink 4 ב-H100?

א. 64 GB/s

ב. 128 GB/s

ג. 3,350 GB/s

ד. 900 GB/s

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

ב-H100 יש 18 links של NVLink 4. כל link מעביר 25 GB/s בכל כיוון: 2 × 25 × 18 = 900 GB/s. כשמשווים מספרים, חשוב לבדוק אם הם לכיוון אחד או לשניים.

למה לא האחרות

64 GB/s: PCIe לכיוון אחד. 128 GB/s: PCIe בשני הכיוונים. 3,350 GB/s: HBM.

רכיבים לחזור עליהם

NVLink

חיבור ישיר ומהיר בין GPU-ים, או בין GPU ל-NVSwitch.

Link

חיבור אחד בין שני צדדים.

איך link אחד מעביר נתונים?

א בכיוון אחד בכל פעם

ב דרך PCIe

ג כשני ערוצים חד-כיווניים שעובדים במקביל

ד דרך ה-L2 של ה-CPU

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

כל Link הוא שני sub-links: אחד לכל כיוון, ושניהם עובדים כל הזמן (full duplex). כל sub-link בנוי מכמה lanes. ב-NVLink 4: 2 lanes בכל כיוון, כל אחד 100 Gb/s. כל Flit מתפזר על כל ה-lanes של ה-sub-link.

למה לא האחרות

אין צורך לסירוגין: לכל כיוון חוטים משלו. NVLink נפרד מ-PCIe.

רכיבים לחזור עליהם

Link

חיבור אחד בין שני צדדים.

Sub-link

הכיוון האחד של link.

Lane

זוג חוטים שמעביר ביטים בכיוון אחד.

SM מבצע load. מה קובע אם הנתונים יבואו מה-HBM המקומי או מ-GPU אחר?

א. פקודה מיוחדת שהמתכנת כותב

ב. ה-Copy engine

ג. ה-page table: הרשומה של הכתובת מצביעה ל-GPU אחר

ד. Fabric Manager בכל גישה

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

זו אותה הוראת load. ה-Driver מיפה מראש את הזיכרון של ה-GPU השכן. כשה-MMU מתרגם, הרשומה ב-Page table אומרת שהכתובת אצל GPU אחר. ה-L2 cache וה-HSHUB שולחים את הבקשה ב-NVLink. לכן GPU שכן נראה כמו עוד זיכרון.

למה לא האחרות

אין פקודה מיוחדת. Copy engine הוא מסלול נפרד להעתקות. Fabric Manager מגדיר ניתוב פעם אחת בהפעלה.

רכיבים לחזור עליהם

Page table

טבלה שממפה כתובות וירטואליות למיקום אמיתי.

MMU

יחידה שמתרגמת כתובות.

HSHUB

הרכזת שמחברת את ה-L2 לקישורים.

איזו יחידה מעבירה בקשה מה-L2 אל הקישורים?

א Front end

ב HSHUB

ג PMU

ד Work distributor

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

ה-HSHUB (High-Speed Hub) יושב בין ה-L2 cache לבלוקי ה-IOCTL. הוא בוחר באיזה link לשלוח כל בקשה. GSP-RM מתכנת אותו כשהקישור עולה. המסלול המלא: SM ← L2 ← HSHUB ← IOCTL ← חוטים.

למה לא האחרות

Front end ו-work distributor מטפלים בפקודות ובחלוקת עבודה. PMU מנהל חשמל.

רכיבים לחזור עליהם

HSHUB

הרכזת שמחברת את ה-L2 לקישורים.

IOCTRL

בלוק החומרה של קבוצת links.

L2 cache

זיכרון מטמון אחד לכל השבב.

מה הבעיה בחיבור GPU 8-ים ישירות זה לזה, בלי מתג?

א כל GPU מחלק את ה-links שלו בין השכנים, אז כל זוג מקבל רק חלק קטן מרוחב הפס

ב אין בעיה

ג NVLink לא עובד בלי מתג

ד PCIe חוסם

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

ל-H100 יש 18 links. חיבור ישיר לשבעה שכנים נותן לכל זוג רק שניים או שלושה. עם NVSwitch, כל GPU מחבר את כל ה-links למתגים, וכל זוג מקבל 900 GB/s מלאים, בקפיצה אחת.

למה לא האחרות

NVLink עובד גם בלי מתג, למשל בין שני GPU-ים. PCIe לא קשור לזה.

רכיבים לחזור עליהם

NVSwitch

שבב מתג שכל ה-GPU-ים מתחברים אליו.

Link

חיבור אחד בין שני צדדים.

בתוך הקישור: הפרוטוקול

השכבות של בלוק הקישור, ואיך הוא מתמודד עם שגיאות ועומס.

רכיבים בפרק: IOCTL · Transaction layer · Data link layer · PHY / SerDes · Flit · CRC · Replay buffer · Credit · Lane · PAM4

מה יש בתוך בלוק IOCTRL?

א. L1-ים ו-SM-ים

ב. HBM ובקר זיכרון

ג. GSP ו-PMU

ד. שכבת טרנזקציות, שכבת קישור, PHY ו-MINION

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

IOCTL הוא בלוק החומרה של קבוצת links. בתוכו: Transaction layer (NVLTLC) שבונה packets, Data link layer (NVLDL) שמוסיף CRC ומנהל replay ו-credits, PHY / SerDes שמוציא ביטים לחוטים, ו-MINION, בקר שמקים את הקישור.

למה לא האחרות

SM-ים נמצאים ב-GPC-ים. HBM ובקרי זיכרון בשולי השבב. GSP ו-PMU בצד הבקרה.

רכיבים לחזור עליהם

IOCTL

בלוק החומרה של קבוצת links.

Transaction layer

השכבה העליונה של הפרוטוקול.

Data link layer

השכבה שהופכת חוט רועש לחוט שנראה בלי שגיאות.

PHY / SerDes

המעגלים ששולחים וקולטים ביטים על החוטים.

MINION

מיקרו-בקר קטן בכל IOCTL, וגם בכל NVSwitch.

איזו שכבה אחראית על CRC ועל שליחה חוזרת (replay)?

א Physical layer

ב Transaction layer

ג Application layer

ד Data link layer

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

ה-Data link layer הופך חוט רועש לחוט שנראה בלי שגיאות: CRC מגלה שגיאות, sequence ו-ack מאשרים קבלה, Replay buffer מאפשר שליחה חוזרת, ו-credits מונעים הצפה. ה-PHY / SerDes רק מעביר ביטים. ה-Transaction layer מניח שהקישור כבר אמין.

למה לא האחרות

ה-Physical לא בודק נכונות. ה-Transaction מקבל קישור אמין. אין Application layer ב-NVLink.

רכיבים לחזור עליהם

Data link layer

השכבה שהופכת חוט רועש לחוט שנראה בלי שגיאות.

CRC

סכום ביקורת שמגלה ביטים שהשתבשו.

Replay buffer

עותק של flits שנשלחו ועוד לא אושרו.

Credit

מייצג מקום פנוי ב-buffer של המקבל.

מה זה flit?

א דגם של GPU

ב סוג של זיכרון

ג יחידת הנתונים הבסיסית שהקישור בודק ושולח שוב אם יש שגיאה

ד פקודה של CUDA

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

Flit (flow control unit) נוצר ב-Transaction layer ונבדק ב-Data link layer. ב-NVLink 1, כפי ש-NVIDIA פרסמה ב-2016: flit של 128 ביט. packet הוא flits, header flit אופציונליים, ועד 16 data flits (256 בתים). ב-header: CRC של 25 ביט, 20 ביט לשכבת הקישור ו-83 ביט לטרנזקציה. בדורות חדשים הפורמט השתנה.

למה לא האחרות

flit הוא לא דגם, לא זיכרון ולא פקודה.

רכיבים לחזור עליהם

Flit

יחידת הנתונים הבסיסית של הקישור.

CRC

סכום ביקורת שמגלה ביטים שהשתבשו.

GPU 1 קיבל flit עם CRC שגוי. מה קורה?

א. הדרייבר מקבל interrupt ושולח מחדש

ב. GPU 1 זורק את ה-flit ואת מה שאחריו, ו-GPU 0 שולח אותם שוב מה-replay buffer, בחומרה

ג. הנתונים הפגומים מגיעים לאפליקציה

ד. הקישור נסגר מיד

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

הכול בחומרה של ה-Data link layer. המקבל זורק את ה-flit הפגום ואת מה שאחריו ומבקש replay. השולח שולח מחדש מה-Replay buffer. המחיר: בערך סיבוב הלוך-חזור. רק אם שגיאות נמשכות ועוברות סף, הקישור נכנס להתאוששות והתוכנה רואה Xid 74.

למה לא האחרות

הדרייבר לא מעורב בשגיאה בודדת. CRC עוצר נתונים פגומים. שגיאה אחת לא סוגרת קישור.

רכיבים לחזור עליהם

Replay buffer

עותק של flits שנשלחו ועוד לא אושרו.

CRC

סכום ביקורת שמגלה ביטים שהשתבשו.

Data link layer

השכבה שהופכת חוט רועש לחוט שנראה בלי שגיאות.

ל-GPU 0 נגמרו ה-credits. מה זה אומר?

א. ה-buffer של המקבל מלא, אז השולח מחכה עד שיחזור credit

ב. הקישור התקלקל

ג. צריך לאתחל את ה-GPU

ד. התעבורה עוברת ל-PCIe

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

Credit מייצג מקום פנוי אצל המקבל. השולח מוציא אחד על כל flit, והמקבל מחזיר כשהוא מפנה מקום. בלי credits השולח מחכה, וכך אין הצפה. בתכנון, ה-buffer צריך לכסות רוחב פס × זמן הלוך-חזור, אחרת הקישור ייעצר גם כשהוא תקין.

למה לא האחרות

זה מצב רגיל של בקרת זרימה, לא תקלה.

רכיבים לחזור עליהם

Credit

מייצג מקום פנוי ב-buffer של המקבל.

Data link layer

השכבה שהופכת חוט רועש לחוט שנראה בלי שגיאות.

למה בדורות חדשים יש פחות lanes בכל link?

א. כי כל lane מהיר הרבה יותר (100–200 Gb/s עם PAM4), אז צריך פחות חוטים

ב. כי רוחב הפס ירד

ג. כי lanes כבר לא נחוצים

ד. כי עברו ל-PCIe

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

מהירות כל Lane: NVLink 1: 8 × 20 Gb/s. NVLink 2: 8 × 25. NVLink 3: 4 × 50. NVLink 4: 2 × 100. NVLink 5: 2 × 200. PAM4 שולח 2 ביטים בכל סמל. החיסכון בחוטים מאפשר יותר links בכל GPU: מ-4 ב-P100 ל-18 ב-H100.

למה לא האחרות

רוחב הפס הכולל עלה בכל דור. lanes עדיין קיימים.

רכיבים לחזור עליהם

Lane

זוג חוטים שמעביר ביטים בכיוון אחד.

PAM4

שיטה לשלוח 2 ביטים בכל סמל במקום ביט אחד.

PHY / SerDes

המעגלים ששולחים וקולטים ביטים על החוטים.

הקמה וניהול: איפה הקוד רץ

מי מקים את הקישור, באיזה סדר, ומה קורה בתקלה.

רכיבים בפרק: NVLink core library · GSP-RM · MINION · Link training · Link states · Xid 74 · nvidia-smi

כשנתונים עוברים ב-NVLink, איזה קוד רץ על כל packet?

א. קוד ב-GSP

ב. קוד ב-MINION

ג. הדרייבר ב-CPU

ד. שום קוד, רק חומרה

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

המסלול SM ← L2 cache ← HSHUB ← Transaction layer ← Data link layer ← PHY / SerDes הוא כולו חומרה. קוד על כל packet היה מאט פי אלפים. הקוד מקים את הקישור, מגדיר אותו, ומגיב לשגיאות ולשינויי חשמל. זו העבודה של מהנדס/ת קושחה.

למה לא האחרות

GSP, MINION וה-Driver מריצים קוד NVLink, אבל רק בהקמה ובתקלות.

רכיבים לחזור עליהם

HSHUB

הרכזת שמחברת את ה-L2 לקישורים.

Transaction layer

השכבה העליונה של הפרוטוקול.

Data link layer

השכבה שהופכת חוט רועש לחוט שנראה בלי שגיאות.

PHY / SerDes

המעגלים ששולחים וקולטים ביטים על החוטים.

מה זה MINION?

א שם של שבב NVSwitch

ב חלק מה-L2

ג בקר Falcon בכל IOCTRL שמריץ microcode להקמה ואימון של הקישור

ד כלי של nvidia-smi

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

כל IOCTRL ב-GPU וב-NVSwitch מכיל MINION: מיקרו-בקר מסוג Falcon. הוא מבצע את הפקודות של Link training: הדלקת ה-PHY / SerDes, הפעלת ה-lanes והפעלת ה-Data link layer. בקוד הפתוח: kernel_minion_gv100.c ו-minion_lr10.c.

למה לא האחרות

NVSwitch הוא שבב שלם; MINION הוא בקר קטן בתוכו. הוא לא חלק מה-L2 ולא כלי תוכנה.

רכיבים לחזור עליהם

MINION

מיקרו-בקר קטן בכל IOCTRL, וגם בכל NVSwitch.

Link training

התהליך שמביא קישור ממצב כבוי למהירות מלאה.

IOCTRL

בלוק החומרה של קבוצת links.

מה הסדר של מצבי הקישור בהקמה?

א. OFF ← SAFE ← HS

ב. HS ← SAFE ← OFF

ג. HS ← OFF ← SAFE

ד. SAFE ← HS ← OFF

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

Link states: OFF (כבוי), SAFE (מצב איטי ל-Link training, הגדרות וגילוי מי בצד השני), HS (מהיר, מעביר תעבורה). שגיאות רבות מחזירות ל-SAFE. השמות מופיעים ב-nvlink.h בקוד הפתוח.

למה לא האחרות

אי אפשר להגיע ל-HS לפני אימון ב-SAFE.

רכיבים לחזור עליהם

Link states

השלבים שהקישור עובר בהקמה.

Link training

התהליך שמביא קישור ממצב כבוי למהירות מלאה.

מה הסדר הנכון של ההקמה, בקיצור?

א. Fabric Manager ← SM ← L2

ב. הדרייבר ← GSP-RM מפעיל MINION ← MINION מאמן ← גילוי הצד השני ← מעבר ל-HS ותכנות HSHUB

ג. MINION ← אפליקציה ← PCIe

ד. שום קוד לא רץ בהקמה

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

בפשטות: ה-Driver שולח RPC ל-GSP-RM. GSP-RM טוען ומפעיל את ה-MINION. MINION מריץ את פקודות ה-Link training, והקישור עולה ל-SAFE. ה-NVLink core library מגלה מי בצד השני ומצמיד את הקצוות. ואז הדרייבר מבקש מ-GSP-RM לעבור ל-HS ולתכנת את ה-HSHUB. הסדר המדויק משתנה לפי דור וגרסה.

למה לא האחרות

האפשרויות האחרות מערבבות יחידות שלא משתתפות בהקמה. ההקמה היא בדיוק המקום שבו הקוד רץ.

רכיבים לחזור עליהם

NVLink core library

קוד בדרייבר שמנהל את כל ה-links.

GSP-RM

החלק של הדרייבר שעבר לרוץ כקושחה על ה-GSP.

MINION

מיקרו-בקר קטן בכל IOCTRL, וגם בכל NVSwitch.

HSHUB

הרכזת שמחברת את ה-L2 לקישורים.

במערכת עם GSP, מי מתכנת בפועל את ה-HSHUB?

א. GSP-RM, אחרי שהדרייבר שולח לו RPC

ב. ה-SM

ג. Fabric Manager

ד. ה-BIOS

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

בקובץ kernel_nvlink.c בקוד הפתוח, הצד של ה-CPU קורא ל-knvlinkExecGspRmRpc, וההערה אומרת שזה RPC ל-GSP-RM כדי לתכנת את רגיסטרי ה-HSHUB. ה-CPU מחליט, ה-GSP מבצע.

למה לא האחרות

ה-SM מריץ קוד משתמש. Fabric Manager מגדיר ניתוב ב-NVSwitch. ה-BIOS לא מתכנת NVLink.

רכיבים לחזור עליהם

GSP-RM

החלק של הדרייבר שעבר לרוץ כקושחה על ה-GSP.

HSHUB

הרכזת שמחברת את ה-L2 לקישורים.

Driver

מודול בקרנל של לינוקס שמנהל את ה-GPU מצד ה-CPU.

שגיאות ממשיכות גם אחרי replay ועוברות סף. מה רואים?

א. Xid 13

ב. שום דבר

ג. תמיד kernel panic

ד. Xid 74 בלוג של הדרייבר

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ד

הסבר מפורט

כש-Replay buffer לא מספיק, הקישור עובר להתאוששות: אימון מחדש, ואם זה לא עוזר, הוא יורד. הדרייבר רושם NVLink Xid 74 (Error. מוני שגיאות: nvidia-smi nvlink -e. תפקיד הקושחה: לקבוע ספים, להחליט מתי לאמן מחדש ומתי להוריד, ולדווח.

למה לא האחרות

Xid 13 הוא שגיאת מנוע גרפי. תקלה כזו כן נרשמת. kernel panic לא קורה בגלל שגיאת קישור.

רכיבים לחזור עליהם

Xid 74

קוד השגיאה שהדרייבר רושם על תקלת NVLink.

nvidia-smi

כלי שמציג מצב של GPU-ים.

Replay buffer

עותק של flits שנשלחו ועוד לא אושרו.

NVSwitch ומערכות גדולות

איך מחברים 8 GPU-ים ויותר, ומה המתג יודע לעשות.

רכיבים בפרק: NVSwitch · NPORT · SOE · Fabric Manager · NVLink SHARP

ב-HGX H100 יש 8 GPU-ים ו-4 שבבי NVSwitch. איך כל GPU מחלק את 18 ה-links שלו?

א. כל 18 לשבב אחד

ב. 6, 6, 6, 0

ג. 5, 4, 4, 5: חלק לכל אחד מארבעת השבבים

ד. 2 לכל GPU אחר

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

5 + 4 + 4 + 5 = 18. כל GPU מחובר לכל ארבעת ה-מתגים, ולכל מתג יש 64 פורטים. כל זוג GPU-ים מקבל 900 GB/s בקפיצה אחת. Fabric Manager מגדיר את הניתוב. החלוקה 5/4/4/5 מגיעה ממקור חיצוני ולא ממסמך של NVIDIA.

למה לא האחרות

מתג אחד היה צוואר בקבוק. 6/6/6/0 משאיר מתג ריק.

רכיבים לחזור עליהם

NVSwitch

שבב מתג שכל ה-GPU-ים מתחברים אליו.

Fabric Manager

תוכנה ב-user space במחשב המארח, במערכות שיש בהן NVSwitch.

מה התפקיד של NPORT ושל SOE בתוך ה-NVSwitch?

א. NPORT: זיכרון · SOE: חישוב

ב. NPORT: ממשק PCIe · SOE: מטמון

ג. NPORT: מנתב packets לפורט של GPU היעד · SOE: בקר שמריץ את הקושחה של המתג

ד. שניהם חלק מה-HBM

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ג

הסבר מפורט

NPORT מקבל packets ושולח כל אחד לפורט שמוביל ל-GPU היעד, לפי טבלאות ש-Fabric Manager הגדיר. SOE הוא בקר שמריץ קושחה במתג, למשל לטמפרטורה ולכבלים. במתג יש גם MINIONs שמאמנים את הקישורים, בדיוק כמו ב-GPU.

למה לא האחרות

למתג אין HBM ואין SM-ים. התפקידים באפשרויות האחרות לא נכונים.

רכיבים לחזור עליהם

NPORT

פורט NVLink בתוך ה-NVSwitch.

SOE

בקר שמריץ קושחה בתוך ה-NVSwitch.

MINION

מיקרו-בקר קטן בכל IOCTRL, וגם בכל NVSwitch.

מה עושה Fabric Manager?

א מריץ kernels

ב מגדיר פעם אחת את הניתוב במתגים במערכות עם NVSwitch

ג מתקן שגיאות CRC

ד מחליף את הדרייבר

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: ב

הסבר מפורט

Fabric Manager היא תוכנה ב-user space במחשב המארח. בהפעלה היא מגדירה ב-NVSwitch איזה פורט מוביל לאיזה GPU. אחרי זה התעבורה זורמת בחומרה, בלי Fabric Manager.

למה לא האחרות

kernels רצים על ה-SM. CRC מטופל ב-Data link layer. Fabric Manager עובד לצד הדרייבר, לא במקומו.

רכיבים לחזור עליהם

Fabric Manager

תוכנה ב-user space במחשב המארח, במערכות שיש בהן NVSwitch.

NVSwitch

שבב מתג שכל ה-GPU-ים מתחברים אליו.

NPORT

פורט NVLink בתוך ה-NVSwitch.

מה עושה NVLink SHARP?

א. המתג סוכם בעצמו את הנתונים ב-all-reduce ושולח את התוצאה לכולם

ב. מצפין את הקישור

ג. מגביר את מהירות ה-lanes

ד. מחליף את ה-L2

לחשוב על תשובה, ואז לדפדף לעמוד הבא.

התשובה הנכונה: א

הסבר מפורט

all-reduce הוא פעולה נפוצה באימון AI: לחבר מערך בין כל ה-GPU-ים. עם NVLink SHARP, כל GPU שולח את החלק שלו פעם אחת ל-NVSwitch, המתג סוכם ושולח את התוצאה לכולם. עוברים פחות נתונים בכל link.

למה לא האחרות

SHARP לא קשור להצפנה, ל-lanes או ל-L2. הוא מעביר חישוב לתוך המתג.

רכיבים לחזור עליהם

NVLink SHARP

יכולת של ה-NVSwitch לחבר נתונים בעצמו.

NVSwitch

שבב מתג שכל ה-GPU-ים מתחברים אליו.

מילון

כל הרכיבים: מה כל אחד עושה במערכת

מסודר לפי אזור בשבב. "מופיע בשלבים" מחזיר לשלבים שמדברים על הרכיב.

המחשב המארח (CPU)

CPU host

המעבד הראשי של המחשב

מה זה

המעבד הרגיל של השרת, שה-GPU מחובר אליו.

מה הוא עושה במערכת

מריץ את האפליקציה, את מערכת ההפעלה ואת ה-Driver. הוא שולח ל-GPU פקודות ונתונים דרך PCIe, ומחכה לסימן שהעבודה הסתיימה.

איפה הוא נמצא

מחוץ ל-GPU. מחובר אליו דרך PCIe.

קשור ל: Driver · PCIe · CUDA

Driver

הדרייבר של NVIDIA (nvidia.ko)

מה זה

מודול בקרנל של לינוקס שמנהל את ה-GPU מצד ה-CPU.

מה הוא עושה במערכת

מקצה זיכרון ובונה page tables, כותב פקודות ל-Pushbuffer ומצלצל ב-Doorbell. בצד ה-NVLink הוא מחזיק את ה-NVLink core library ושולח בקשות (RPC) ל-GSP-RM, שמבצע אותן על החומרה.

איפה הוא נמצא

רץ על ה-CPU, בקרנל.

`open-gpu-kernel-modules` `CPU-RM ↔ GSP-RM`

קשור ל: GSP-RM · NVLink core library · Page table · Pushbuffer

מופיע בשלבים: 5 · 22 · 38

CUDA

פלטפורמת התכנות של NVIDIA

מה זה

הספריות וה-runtime שבעזרתם כותבים קוד ל-GPU.

מה הוא עושה במערכת

מתרגמת קריאות של האפליקציה, כמו cudaMemcpy או הרצה של Kernel, לבקשות ל-Driver.

איפה הוא נמצא

רצה על ה-CPU, בתוך האפליקציה.

`cudaMemcpy` `kernel launch`

קשור ל: Kernel · Driver

Fabric Manager

מנהל הרשת של NVSwitch

מה זה

תוכנה ב-user space במחשב המארח, במערכות שיש בהן NVSwitch.

מה הוא עושה במערכת

מגדירה פעם אחת, בהפעלה, את הניתוב במתגים: איזה פורט מוביל לאיזה GPU. היא לא משתתפת בכל גישה לזיכרון.

איפה הוא נמצא

רצה על ה-CPU, ב-user space.

קשור ל: NVSwitch · NPORT

מופיע בשלבים: 40 · 42

nvidia-smi

כלי ניטור בשורת פקודה

מה זה

כלי שמציג מצב של GPU-ים.

מה הוא עושה במערכת

בצד ה-NVLink: nvidia-smi nvlink -s מציג את מצב ה-links, ו- nvidia-smi nvlink -e מציג מוני שגיאות. שימושי לבדוק את עבודת הקושחה.

איפה הוא נמצא

רץ על ה-CPU.

`nvidia-smi nvlink -s` `nvidia-smi nvlink -e`

קשור ל: Xid 74 · CRC

מופיע בשלבים: 39

בקרה: איך עבודה נכנסת

PCIe ממשק PCI Express

מה זה

החיבור בין ה-GPU ל-CPU.

מה הוא עושה במערכת

דרכו עוברים: פקודות מה-Driver, העתקות של Copy engine, בקשות RPC ל-GSP-RM, וקריאות של ה-CPU לרגיסטרים של ה-GPU. הוא איטי בהרבה מ-NVLink, ולכן לא משמש להחלפת נתונים כבדה בין GPU-ים.

איפה הוא נמצא

בראש ה-GPU, בצד הבקרה.

`Gen5 ×16` `לכל כיוון 64 GB/s`

`בשני הכיוונים 128 GB/s`

קשור ל: CPU host · Front end · Copy engine

מופיע בשלבים: 4 · 8 · 22

Front end

היחידה שקוראת פקודות

מה זה

היחידה בראש ה-GPU שקוראת את רשימת הפקודות.

מה הוא עושה במערכת

אחרי שה-Driver מצלצל ב-Doorbell, ה-front end שולף פקודות מה-Pushbuffer, מפענח אותן ומעביר כל אחת ליחידה הנכונה: עבודת חישוב ל-Work distributor, העתקות ל-Copy engine. בסוף הוא כותב Semaphore כדי לסמן שהעבודה הסתיימה.

איפה הוא נמצא

בראש ה-GPU, ליד PCIe.

קשור ל: Pushbuffer · Doorbell · Work distributor · Semaphore

מופיע בשלבים: 6 · 9

Pushbuffer

buffer של פקודות

מה זה

אזור בזיכרון שבו הדרייבר כותב רשימת פקודות ל-GPU.

מה הוא עושה במערכת

זו "רשימת המשימות" של ה-GPU. ה-Driver כותב אליה, וה-Front end קורא ממנה.

איפה הוא נמצא

בזיכרון (של ה-CPU או של ה-GPU).

קשור ל: Front end · Doorbell

מופיע בשלבים: 5

Doorbell

רגיסטר "פעמון"

מה זה

רגיסטר שהדרייבר כותב אליו.

מה הוא עושה במערכת

כתיבה אליו אומרת ל-Front end: "יש פקודות חדשות ב-Pushbuffer". כך ה-GPU לא צריך לבדוק כל הזמן.

איפה הוא נמצא

רגיסטר ב-GPU, שה-CPU כותב אליו דרך PCIe.

קשור ל: Front end · Pushbuffer

מופיע בשלבים: 5

Work distributor

מחלק העבודה (GigaThread engine)

מה זה

החלק של ה-front end שמחלק עבודת חישוב.

מה הוא עושה במערכת

מקבל Grid של thread blocks ושולח כל block ל-SM שיש בו מקום. כשבלוק מסתיים, הוא שולח את הבא.

איפה הוא נמצא

בצד הבקרה, בין ה-front end ל-GPC-ים.

קשור ל: Front end · Thread block · SM

מופיע בשלבים: 6 · 7

Copy engine מנוע העתקה (DMA)

מה זה

מנוע שמעתיק בלוקים גדולים של זיכרון.

מה הוא עושה במערכת

מעביר נתונים בין זיכרון המחשב ל-HBM דרך PCIe, ובין GPU-ים דרך NVLink, בלי להעסיק את ה-SM-ים. לכן העתקה וחישוב יכולים לרוץ במקביל. cudaMemcpy משתמש בו.

איפה הוא נמצא

בצד הבקרה של ה-GPU.

`DMA` `cudaMemcpy`

קשור ל: PCIe · HBM · NVLink

מופיע בשלבים: 6 · 8

Semaphore סימן סיום

מה זה

כתיבה של ערך לזיכרון כדי לסמן שמשהו הסתיים.

מה הוא עושה במערכת

כשכל ה-blocks של Kernel מסתיימים, ה-Front end כותב semaphore. ה-CPU רואה את הערך ויודע שהתוצאות מוכנות.

איפה הוא נמצא

ערך בזיכרון שה-CPU וה-GPU רואים.

קשור ל: Front end · CPU host

מופיע בשלבים: 9

תוכנה: מה שמתכנתים כותבים

Kernel פונקציה שרצה על ה-GPU

מה זה פונקציה שמתכנתים כותבים ב-CUDA, ורצה על ה-GPU באלפי עותקים.

מה הוא עושה במערכת הרצה אחת של kernel נקראת Grid. כל עותק הוא Thread, וכל אחד עובד על פיסת נתונים אחרת.

איפה הוא נמצא קוד שרץ על ה-SM-ים.

קשור ל: Grid · Thread · CUDA

Grid הרצה אחת של kernel

מה זה כל ה-thread blocks של הרצה אחת של Kernel.

מה הוא עושה במערכת הגריד רץ על כל ה-GPU: ה-Work distributor מפזר את ה-blocks שלו על כל ה-SM-ים.

איפה הוא נמצא כל ה-GPU.

קשור ל: Thread block · Work distributor

מופיע בשלבים: 7

Thread block

בלוק של threads

מה זה

קבוצה של עד 1,024 threads.

מה הוא עושה במערכת

בלוק רץ על SM אחד ונשאר בו עד שהוא מסתיים. לכן ה-threads שלו יכולים לחלוק shared memory ולהסתנכרן.

איפה הוא נמצא

SM אחד.

`עד 1,024 threads`

קשור ל: SM · Warp · L1 / shared memory

מופיע בשלבים: 7 · 13

Warp

קבוצה של 32 threads

מה זה

32 threads שמריצים תמיד את אותה הוראה יחד.

מה הוא עושה במערכת

יחידת העבודה האמיתית של החומרה. ה-Warp scheduler בוחר warp מוכן בכל מחזור ושולח הוראה אחת ל-32 הנתיבים שלו. warp רץ על SM partition אחד.

איפה הוא נמצא

SM partition אחד.

`32 threads`

קשור ל: Warp scheduler · SM partition · Thread

מופיע בשלבים: 1 · 12 · 14

Thread

עותק אחד של ה-kernel

מה זה

עותק אחד של ה-Kernel, שעובד על פיסת נתונים אחת.

מה הוא עושה במערכת

כל thread רץ על נתיב אחד, ויש לו רגיסטרים פרטיים ב-Register file.

איפה הוא נמצא

נתיב אחד ב-SM.

קשור ל: Warp · FP32 lanes · Register file

מופיע בשלבים: 12

חישוב: GPC, SM ומה שבתוכם

GPC

Graphics Processing Cluster

מה זה

קבוצה גדולה של יחידות חישוב.

מה הוא עושה במערכת

מקבצת TPC-ים, ובתוכם את ה-SM-ים. העבודה מגיעה אליה מה-Work distributor.

איפה הוא נמצא

באמצע השבב. ב-GH100 המלא יש 8 GPC בכל 9 TPC.

קשור ל: TPC · SM

מופיע בשלבים: 2 · 10

TPC Texture Processing Cluster

מה זה זוג SM-ים.

מה הוא עושה במערכת יחידת ביניים בהירכיה: GPC ← TPC ← SM. בכל TPC יש 2 SM.

איפה הוא נמצא בתוך GPC.

`2 SM` `בשבב המלא 72 TPC`

קשור ל: GPC · SM

מופיע בשלבים: 2 · 10

SM Streaming Multiprocessor

מה זה הליבה של ה-GPU, שמריצה את ה-threads.

מה הוא עושה במערכת מקבל thread blocks ומריץ את ה-warps שלהם. בתוכו 4 partitions ו-256 KB של L1 / shared memory.

איפה הוא נמצא בתוך TPC. ב-H100 SXM5 פעילים 132 (בשבב המלא 144).

`פעילים 132 SM` `128 FP32 lanes` `4 tensor cores`

קשור ל: TPC · SM partition · L1 / shared memory · Thread block

מופיע בשלבים: 1 · 2 · 10 · 13 · 15 · 17

SM partition

רבע של SM

מה זה

אחד מארבעת החלקים של SM.

מה הוא עושה במערכת

בכל partition יש Warp scheduler, Register file של 64 KB, 32 נתיבי FP32 ו-Tensor core אחד. כל warp רץ על partition אחד.

איפה הוא נמצא

בתוך 4 SM. בכל SM.

`64 KB registers` `32 FP32` `1 tensor core`

קשור ל: Warp scheduler · Register file · FP32 lanes · Tensor core

מופיע בשלבים: 10 · 11

Warp scheduler

מתזמן ה-warps

מה זה

היחידה שבוחרת איזה warp ירוץ עכשיו.

מה הוא עושה במערכת

מחזיקה הרבה warps. כש-warp אחד מחכה לזיכרון, היא עוברת מיד לאחר, וכך הנתיבים לא עומדים. זה "הסתרת השהיה" (latency hiding).

איפה הוא נמצא

אחד בכל SM partition.

`latency hiding`

קשור ל: Warp · SM partition

מופיע בשלבים: 11 · 12 · 14

Register file קובץ הרגיסטרים

מה זה האחסון המהיר ביותר על השבב.

מה הוא עושה במערכת מחזיק את המשתנים הפרטיים של כל Thread. כיוון שהרגיסטרים של כל ה-warps נמצאים בו כל הזמן, המעבר בין warps כמעט לא עולה כלום.

איפה הוא נמצא 64 KB בכל partition, 256 KB בכל SM.

`256 KB / SM`

קשור ל: SM partition · Warp scheduler

מופיע בשלבים: 11 · 14 · 19

FP32 lanes נתיבי חישוב (CUDA cores)

מה זה יחידות שמבצעות חישוב רגיל במספרים.

מה הוא עושה במערכת כל נתיב מריץ Thread אחד. 32 נתיבים מבצעים יחד הוראה אחת של Warp.

איפה הוא נמצא 32 בכל partition, 128 בכל SM.

`128 / SM` `≈16,900 ב-H100`

קשור ל: SM partition · Thread

מופיע בשלבים: 1 · 11

Tensor core

ליבת מטריצות

מה זה

יחידה שמכפילה מטריצות קטנות בצעד אחד.

מה הוא עושה במערכת

עושה את רוב העבודה באימון והרצה של מודלי AI, שבנויים מכפל מטריצות.

איפה הוא נמצא

אחד בכל 4 partition, בכל SM.

`4 / SM`

קשור ל: SM partition · SM

מופיע בשלבים: 11 · 15

זיכרון

L1 / shared memory

זיכרון מהיר של ה-SM

מה זה

זיכרון מהיר בתוך כל SM.

מה הוא עושה במערכת

חלק ממנו משמש cache, וחלק משמש shared memory: "דף טיוטה" שה-threads של Thread block אחד חולקים. גישה ל-L1 מהירה בהרבה מגישה ל-L2 cache.

איפה הוא נמצא

בתוך כל SM. 256 KB.

`256 KB / SM`

קשור ל: SM · L2 cache · Thread block

מופיע בשלבים: 3 · 13 · 18 · 19

MMU

Memory Management Unit

מה זה

יחידה שמתרגמת כתובות.

מה הוא עושה במערכת

תוכנות עובדות עם כתובות וירטואליות. ה-MMU מוצא את המיקום האמיתי בעזרת page tables, וחוסם גישה אסורה. רשומה יכולה להצביע לזיכרון של GPU אחר, וכך load רגיל יוצא ב-NVLink.

איפה הוא נמצא

במסלול מה-SM לזיכרון.

`virtual → physical`

קשור ל: Page table · L2 cache · NVLink

מופיע בשלבים: 16 · 25

Page table

טבלת דפים

מה זה

טבלה שממפה כתובות וירטואליות למיקום אמיתי.

מה הוא עושה במערכת

ה-Driver בונה אותה. ה-MMU קורא ממנה. היא קובעת אם כתובת נמצאת ב-HBM המקומי או אצל GPU שכן.

איפה הוא נמצא

בזיכרון.

קשור ל: MMU · Driver

מופיע בשלבים: 16 · 25

Crossbar

רשת פנימית

מה זה

הרשת שמחברת את ה-SM-ים לזיכרון.

מה הוא עושה במערכת

מחברת כל SM לכל חלק (slice) של ה-L2 cache, כך שכל SM יכול להגיע לכל כתובת.

איפה הוא נמצא

בין ה-GPC-ים ל-L2.

קשור ל: SM · L2 cache

מופיע בשלבים: 17 · 18

L2 cache

זיכרון מטמון משותף

מה זה

זיכרון מטמון אחד לכל השבב.

מה הוא עושה במערכת

נקודת המפגש: כל גישה לזיכרון, מקומית או מרוחקת, עוברת בו. בפספוס, בקשה מקומית ממשיכה ל-Memory controller, ובקשה לזיכרון של GPU אחר ממשיכה ל-HSHUB. גם בקשות מ-GPU שכן מקבלות תשובה מה-L2.

איפה הוא נמצא

באמצע השבב, בשני חצאים.

`50 MB`

קשור ל: Crossbar · Memory controller · HSHUB

מופיע בשלבים: 3 · 17 · 18 · 19 · 26

Memory controller בקר זיכרון

מה זה יחידה שקוראת וכותבת ל-HBM.

מה הוא עושה במערכת מקבלת בקשות מה-L2 cache שלא נמצאו בו, ומדברת עם שבבי ה-HBM.

איפה הוא נמצא בשולי השבב, ליד ה-HBM.

קשור ל: L2 cache · HBM

מופיע בשלבים: 3 · 18

HBM High Bandwidth Memory

מה זה הזיכרון הגדול של ה-GPU.

מה הוא עושה במערכת מחזיק את הנתונים של התוכנית. גדול ומהיר, אבל איטי יותר מה-L2 cache. ה-Copy engine ממלא אותו.

איפה הוא נמצא שבבים מוערמים ליד ה-GPU. ב-H100 SXM5 יש 5 פעילים.

`80 GB` `3.35 TB/s`

קשור ל: Memory controller · L2 cache

מופיע בשלבים: 3 · 4 · 8 · 19

קושחה על ה-GPU

GSP

GPU System Processor

מה זה

מעבד RISC-V בתוך ה-GPU.

מה הוא עושה במערכת

מריץ את GSP-RM, הקושחה שמאתחלת ומנהלת את השבב. זה ה"בית" של מהנדס/ת קושחה.

איפה הוא נמצא

בצד הבקרה של ה-GPU.

`RISC-V`

קשור ל: GSP-RM · PMU · SEC2 · FSP

מופיע בשלבים: 20

GSP-RM

Resource Manager שרץ על ה-GSP

מה זה

החלק של הדרייבר שעבר לרוץ כקושחה על ה-GSP.

מה הוא עושה במערכת

ה-Driver ב-CPU מחליט ושולח RPC, ו-GSP-RM מבצע על החומרה. למשל, הוא מתכנת את רגיסטרי ה-HSHUB כשהקישור עולה.

איפה הוא נמצא

רץ על ה-GSP.

`gsp_*.bin` `knvlinkExecGspRmRpc`

קשור ל: GSP · Driver · HSHUB

מופיע בשלבים: 20 · 22 · 37 · 38

PMU Power Management Unit

מה זה בקר קטן לניהול צריכת חשמל.

מה הוא עושה במערכת מנהל שעונים ומתחים לפי העומס והטמפרטורה.

איפה הוא נמצא בצד הבקרה.

קשור ל: GSP

מופיע בשלבים: 21

SEC2 בקר אבטחה

מה זה בקר קטן למשימות אבטחה.

מה הוא עושה במערכת מטפל בפעולות שדורשות אמון, כמו טעינה ובדיקה של קושחה.

איפה הוא נמצא בצד הבקרה.

קשור ל: GSP FSP

מופיע בשלבים: 21

FSP Firmware Security Processor

מה זה בקר אבטחה ב-Hopper.

מה הוא עושה במערכת אחראי על אתחול מאובטח: בודק שהקושחה שנטענת אמיתית, לפני שהיא רצה (root of trust).

איפה הוא נמצא בצד הבקרה.

קשור ל: GSP SEC2

מופיע בשלבים: 21

NVLink: הקישור בין GPU-ים

NVLink

הקישור המהיר של NVIDIA בין GPU-ים

מה זה

חיבור ישיר ומהיר בין GPU-ים, או בין GPU ל-NVSwitch.

מה הוא עושה במערכת

מעביר פעולות זיכרון רגילות (read, write, atomic) בין GPU-ים, כך ש-GPU שכן נראה כמו עוד זיכרון. בנוי מ-links.

איפה הוא נמצא

יוצא מה-HSHUB דרך בלוקי IOCTL.

`NVLink 4: 18 links` `900 GB/s`

קשור ל: Link · HSHUB · IOCTL · NVSwitch

מופיע בשלבים: 4 · 23

Link

קישור אחד

מה זה

חיבור אחד בין שני צדדים.

מה הוא עושה במערכת

כל link הוא שני sub-links חד-כיווניים שעובדים במקביל. ב-NVLink 4 כל link מעביר 25 GB/s בכל כיוון.

איפה הוא נמצא

בין שני GPU-ים, או בין GPU למתג.

`25 GB/s לכל כיוון (NVLink 4)`

קשור ל: Sub-link · Lane

מופיע בשלבים: 23 · 24 · 27

Sub-link
חצי קישור, כיוון אחד

מה זה
הכיוון האחד של link.

מה הוא עושה במערכת
מורכב מכמה lanes. כל Flit מתפזר על כל ה-lanes של ה-sub-link שלו.

איפה הוא נמצא
בתוך link. 2 בכל link.

קשור ל: Link · Lane · Flit

מופיע בשלבים: 24

Lane
נתיב פיזי

מה זה
זוג חוטים שמעביר ביטים בכיוון אחד.

מה הוא עושה במערכת
הרבה lanes יחד יוצרים Sub-link. בכל דור כל lane מהיר יותר (בזכות PAM4), אז צריך פחות lanes.

איפה הוא נמצא
בשכבה הפיזית, יוצא מה-PHY / SerDes.

`NVLink 1: 8 × 20 Gb/s` `NVLink 4: 2 × 100 Gb/s` `NVLink 5: 2 × 200 Gb/s`

קשור ל: Sub-link · PHY / SerDes · PAM4

מופיע בשלבים: 24 · 33

PAM4 אפנון בארבע רמות

מה זה

שיטה לשלוח 2 ביטים בכל סמל במקום ביט אחד.

מה הוא עושה במערכת

מכפילה את המהירות של כל Lane באותו קצב סמלים. משמשת מ-NVLink 4.

איפה הוא נמצא

ב-PHY / SerDes.

`2 bits / symbol`

קשור ל: Lane · PHY / SerDes

מופיע בשלבים: 33

HSHUB High-Speed Hub

מה זה

הרכזת שמחברת את ה-L2 לקישורים.

מה הוא עושה במערכת

כשה-L2 cache מקבל בקשה לזיכרון של GPU אחר, ה-HSHUB בוחר באיזה link לשלוח אותה. GSP-RM מתכנת אותו כשהקישור עולה.

איפה הוא נמצא

בין ה-L2 לבלוקי ה-IOCTL.

קשור ל: L2 cache · IOCTL · GSP-RM

מופיע בשלבים: 18 · 25 · 26 · 34 · 37 · 38

IOCTRL

בלוק קישור

מה זה

בלוק החומרה של קבוצת links.

מה הוא עושה במערכת

מכיל את שכבות הקישור: Transaction layer, Data link layer ו-PHY / SerDes, וגם MINION שמקים את הקישור.

איפה הוא נמצא

בשולי השבב, אחרי ה-HSHUB.

קשור ל: Transaction layer · Data link layer · PHY / SerDes · MINION

מופיע בשלבים: 26 · 28 · 35

Transaction layer

שכבת הטרנזקציות (NVLTLC)

מה זה

השכבה העליונה של הפרוטוקול.

מה הוא עושה במערכת

הופכת בקשת זיכרון (read, write, atomic) ל-packet של flits, ובצד השני מבצעת את הבקשה מול ה-L2.

איפה הוא נמצא

בתוך IOCTRL.

NVLTLC

קשור ל: Data link layer · Flit · IOCTRL

מופיע בשלבים: 28 · 34

Data link layer

שכבת הקישור (NVLDL)

מה זה

השכבה שהופכת חוט רועש לחוט שנראה בלי שגיאות.

מה הוא עושה במערכת

מוסיפה CRC ומספר sequence לכל flit, שומרת עותק ב-Replay buffer, ומנהלת credits. כל זה בחומרה.

איפה הוא נמצא

בתוך IOCTRL, בין ה-TL ל-PHY.

`NVLDL`

קשור ל:

CRC · Replay buffer · Credit · Transaction layer · PHY / SerDes

מופיע בשלבים: 28 · 29 · 31 · 32 · 34

PHY / SerDes

השכבה הפיזית

מה זה

המעגלים ששולחים וקולטים ביטים על החוטים.

מה הוא עושה במערכת

הופכת flits לזרם ביטים מהיר על ה-lanes (serialize), ובצד השני משחזרת שעון ומיישרת את ה-lanes. צריכה Link training לפני שהיא עובדת במהירות מלאה.

איפה הוא נמצא

בתחתית ה-IOCTRL, מחוברת לחוטים.

`SerDes`

קשור ל:

Lane · PAM4 · Link training

מופיע בשלבים: 28 · 33 · 34

Flit

flow control unit

מה זה

יחידת הנתונים הבסיסית של הקישור.

מה הוא עושה במערכת

זו היחידה שהקישור בודק עם CRC ושולח שוב אם יש שגיאה. ב-NVLink 1: flit של 128 ביט, ו-packet של header ועוד עד 256) 16 data flits בתים).

איפה הוא נמצא

נוצר ב-TL, נבדק ב-DL.

`NVLink 1: 128 bit` `256 B payload`

קשור ל: Transaction layer · Data link layer · CRC

מופיע בשלבים: 30

CRC

בדיקת שגיאות

מה זה

סכום ביקורת שמגלה ביטים שהשתבשו.

מה הוא עושה במערכת

ה-Data link layer מחשב CRC לכל flit. אם המקבל מוצא CRC שגוי, הוא זורק את ה-flit ומבקש Replay buffer. ב-NVLink 1 ה-CRC הוא 25 ביט בתוך ה-header.

איפה הוא נמצא

ב-DL.

`NVLink 1: 25 bit`

קשור ל: Data link layer · Replay buffer · Flit

מופיע בשלבים: 29 · 30 · 31

Replay buffer

buffer לשליחה חוזרת

מה זה

עותק של flits שנשלחו ועוד לא אושרו.

מה הוא עושה במערכת

כשהמקבל מוצא CRC שגוי, השולח שולח מחדש מה-buffer את ה-flit הפגום ואת כל מה שאחריו. הכול בחומרה, בלי תוכנה.

איפה הוא נמצא

בצד השולח, ב-DL.

```
מחיר: בערך סיבוב אחד הלוך-חזור
```

קשור ל: CRC · Data link layer

מופיע בשלבים: 29 · 31 · 39

Credit

אישור מקום פנוי

מה זה

מייצג מקום פנוי ב-buffer של המקבל.

מה הוא עושה במערכת

השולח מוציא credit על כל flit, והמקבל מחזיר credit כשהוא מפנה מקום. כשאין credits השולח מחכה. כך אין הצפה ואין אובדן נתונים.

איפה הוא נמצא

ב-DL, בשני הצדדים.

```
buffer ≥ bandwidth × round trip
```

קשור ל: Data link layer

מופיע בשלבים: 29 · 32

MINION

בקר Falcon בבלוק הקישור

מה זה

מיקרו-בקר קטן בכל IOCTRL, וגם בכל NVSwitch.

מה הוא עושה במערכת

מריץ microcode שמבצע את הפקודות הנמוכות של Link training: הדלקת ה-PHY / SerDes, הפעלת ה-lanes והפעלת ה-Data link layer. GSP-RM או הדרייבר טוענים ומפעילים אותו.

איפה הוא נמצא

בתוך IOCTRL.

`Falcon` `kernel_minion_gv100.c`

קשור ל: IOCTRL · Link training · GSP-RM

מופיע בשלבים: 28 · 35 · 37 · 41

Link states

מצבי הקישור

מה זה

השלבים שהקישור עובר בהקמה.

מה הוא עושה במערכת

OFF: כבוי. SAFE: מצב איטי ל-Link training, הגדרות וגילוי מי מחובר בצד השני. HS (High Speed): מעביר תעבורה. שגיאות רבות מחזירות ל-SAFE לאימון מחדש.

איפה הוא נמצא

מוגדרים ב-nvlink.h בקוד הפתוח.

`OFF → SAFE → HS`

קשור ל: Link training · MINION · NVLink core library

מופיע בשלבים: 36

Link training אימון הקישור

מה זה

התהליך שמביא קישור ממצב כבוי למהירות מלאה.

מה הוא עושה במערכת

אלגוריתם של ה-SerDes, כיוון

NVSwitch מתג NVLink

מה זה

שבב מתג שכל ה-GPU-ים מתחברים אליו.

מה הוא עושה במערכת

נותן לכל זוג GPU-ים את מלוא רוחב הפס, עם קפיצה אחת. בתוכו NPORTs, MINION ו-SOE.

איפה הוא נמצא

על הלוח, בין ה-GPU-ים. ב-HGX H100 יש 4.

פורטים (דור 3) 64

קשור ל: NPORT · SOE · Fabric Manager · NVLink SHARP

מופיע בשלבים: 27 · 40 · 42 · 43

NPORT פורט של המתג

מה זה

פורט NVLink בתוך ה-NVSwitch.

מה הוא עושה במערכת

מקבל packets ומנתב אותם לפורט שמוביל ל-GPU היעד, לפי טבלאות ש-Fabric Manager הגדיר.

איפה הוא נמצא

בתוך NVSwitch.

קשור ל: NVSwitch · Fabric Manager

מופיע בשלבים: 41 · 42

SOE

הבקר של ה-NVSwitch

מה זה

בקר שמריץ קושחה בתוך ה-NVSwitch.

מה הוא עושה במערכת

מטפל במשימות של המתג, למשל טמפרטורה וניהול כבלים. הדרייבר ב-CPU מאתחל אותו.

איפה הוא נמצא

בתוך NVSwitch.

קשור ל: NVSwitch

מופיע בשלבים: 41

NVLink SHARP

חישוב בתוך המתג

מה זה

יכולת של ה-NVSwitch לחבר נתונים בעצמו.

מה הוא עושה במערכת

ב-all-reduce כל GPU שולח את החלק שלו פעם אחת, ה-NVSwitch סוכם ושולח את התוצאה לכולם. כך עוברים פחות נתונים בכל link.

איפה הוא נמצא

בתוך NVSwitch.

`in-network reduction`

קשור ל: NVSwitch

מופיע בשלבים: 43